STT 811 Project

```r
grocery <- read.csv("grocery_inventory_clean.csv")
```

```r
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```r
library(ggplot2)
library(ggiraph)
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```r
library(xgboost)
library(randomForest)
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```r
library(nnet)
library(dplyr)
```

```r
grocery$FIFO_FEFO <- as.factor(grocery$FIFO_FEFO)
grocery$Inventory_Status <- as.factor(grocery$Inventory_Status)
grocery$ABC_Class <- as.factor(grocery$ABC_Class)
stock_variables <- c("Quantity_On_Hand", "Quantity_Reserved", "Quantity_Committed", "Damaged_Qty", "Retu
"Stock_Age_Days")
planning_variables <- c("Avg_Daily_Sales", "Forecast_Next_30d", "Days_of_Inventory", "Reorder_Point", "S
financial_variables <- c("Unit_Cost_USD", "Last_Purchase_Price_USD", "Total_Inventory_Value_USD", "Suppl
grocery_stock_status <- grocery[c(stock_variables, 'ABC_Class')]
grocery_planning_velocity <- grocery[c(planning_variables, 'ABC_Class')]
grocery_financial_quality <- grocery[c(financial_variables, 'ABC_Class')]
```

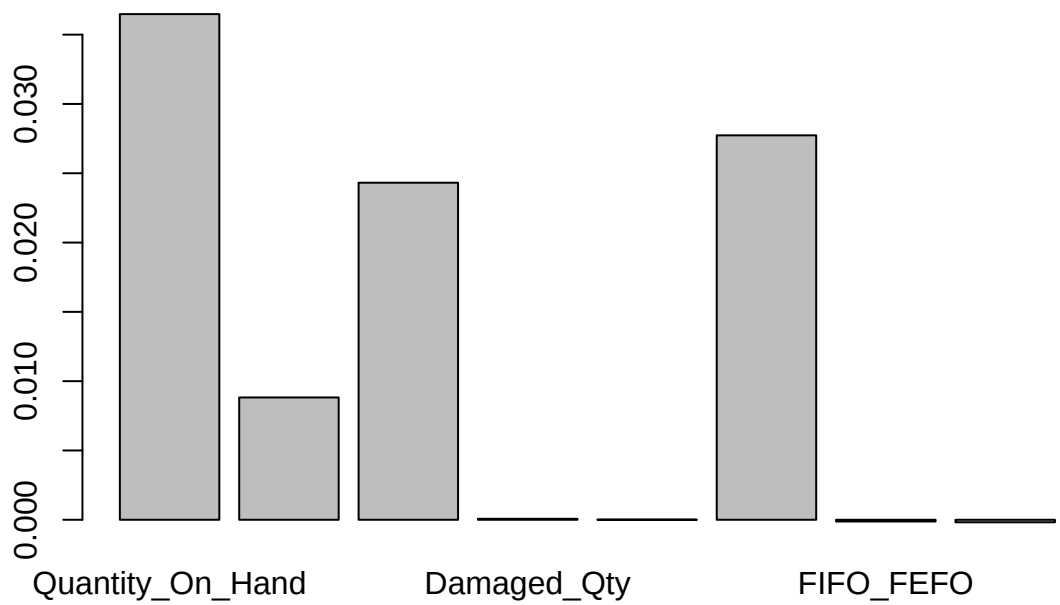
```r
split_pct <- 0.8
n <- length(grocery$ABC_Class)*split_pct
row_samp <- sample(1:length(grocery$ABC_Class), n, replace = FALSE)
train <- grocery[row_samp,]
test <- grocery[-row_samp,]
train_stock_status <- grocery_stock_status[row_samp,]
test_stock_status <- grocery_stock_status[-row_samp,]
train_planning_velocity <- grocery_planning_velocity[row_samp,]
test_planning_velocity <- grocery_planning_velocity[-row_samp,]
train_financial_quality <- grocery_financial_quality[row_samp,]
test_financial_quality <- grocery_financial_quality[-row_samp,]
```

```r
grocery_stock_rf <- randomForest(ABC_Class ~ . - ABC_Class,
                      data = train_stock_status, mtry = 2,
                      importance = TRUE, ntree = 1000, maxnodes = 4)
rf_predict_stock <- predict(grocery_stock_rf, test_stock_status)
confusionMatrix(rf_predict_stock, as.factor(test_stock_status$ABC_Class))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  A  B  C
##          A  0  0  0
##          B 18  7  9
##          C 21 52 93
##
## Overall Statistics
##
##                Accuracy : 0.5
##                  95% CI : (0.4287, 0.5713)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : 0.6383
##
##                   Kappa : 0.0504
##
##  Mcnemar's Test P-Value : 5.994e-15
##
## Statistics by Class:
```

```
##
##                      Class: A Class: B Class: C
## Sensitivity             0.000   0.1186   0.9118
## Specificity             1.000   0.8085   0.2551
## Pos Pred Value            NaN   0.2059   0.5602
## Neg Pred Value          0.805   0.6867   0.7353
## Prevalence              0.195   0.2950   0.5100
## Detection Rate          0.000   0.0350   0.4650
## Detection Prevalence    0.000   0.1700   0.8300
## Balanced Accuracy       0.500   0.4636   0.5834
```

```r
barplot(grocery_stock_rf$importance[,3])
```

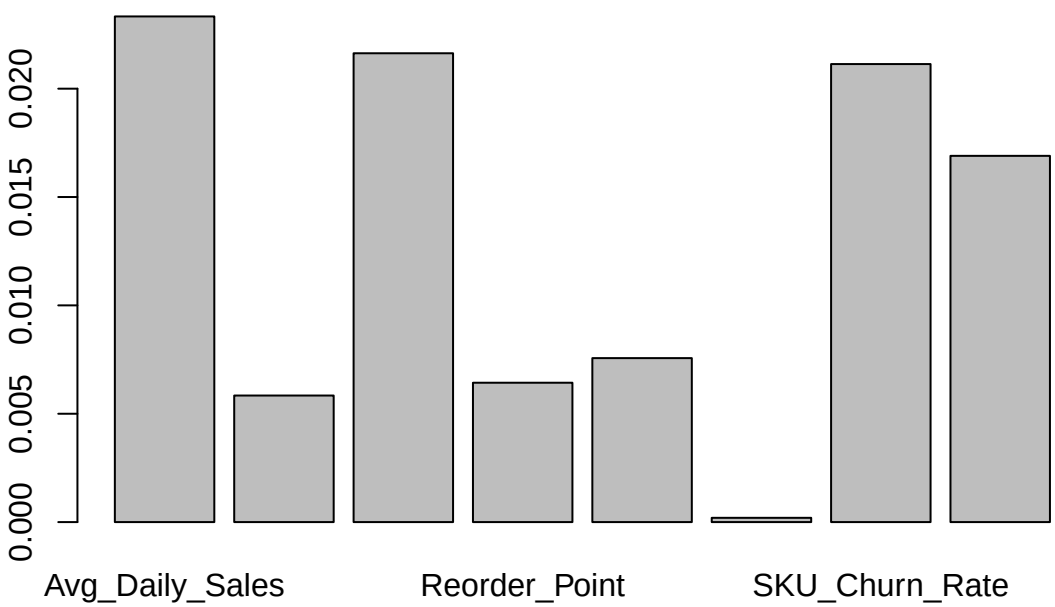



```r
grocery_planning_rf <- randomForest(ABC_Class ~ . - ABC_Class,
                      data = train_planning_velocity, mtry = 2,
                      importance = TRUE, ntree = 1000, maxnodes = 4)
rf_predict_planning <- predict(grocery_planning_rf,  test_planning_velocity)
confusionMatrix(rf_predict_planning, as.factor(test_planning_velocity$ABC_Class))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   A   B   C
##          A   0   0   0
##          B  16  11   1
```

```
##          C  23  48 101
##
## Overall Statistics
##
##                Accuracy : 0.56
##                  95% CI : (0.4883, 0.6299)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : 0.08938
##
##                   Kappa : 0.154
##
##  Mcnemar's Test P-Value : < 2e-16
##
## Statistics by Class:
##
##                      Class: A Class: B Class: C
## Sensitivity             0.000   0.1864   0.9902
## Specificity             1.000   0.8794   0.2755
## Pos Pred Value            NaN   0.3929   0.5872
## Neg Pred Value          0.805   0.7209   0.9643
## Prevalence              0.195   0.2950   0.5100
## Detection Rate          0.000   0.0550   0.5050
## Detection Prevalence    0.000   0.1400   0.8600
## Balanced Accuracy       0.500   0.5329   0.6329
```

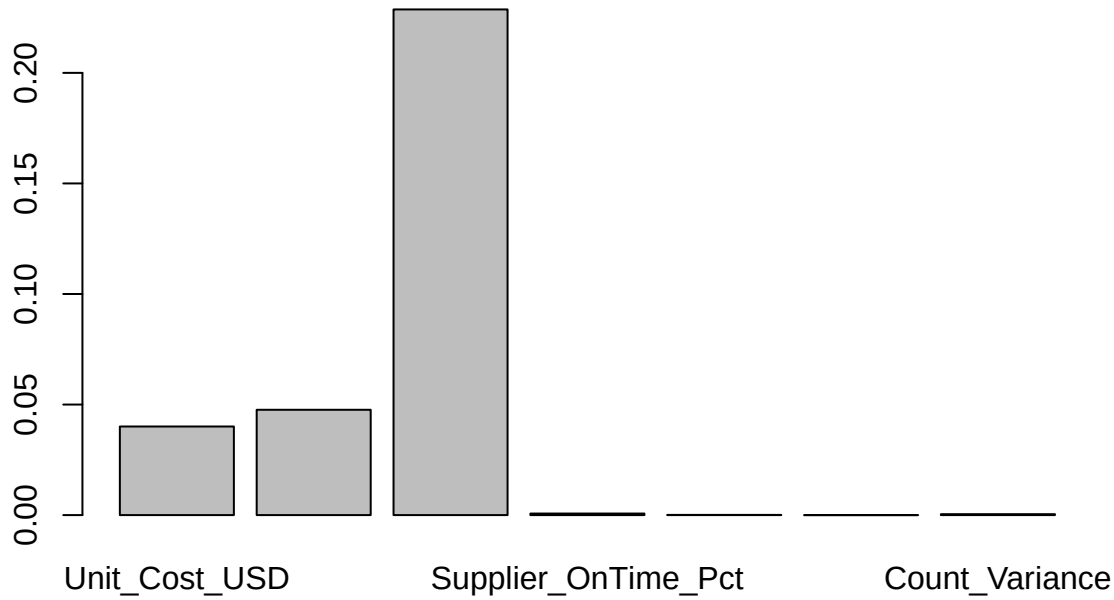
```r
barplot(grocery_planning_rf$importance[,3])
```



```r
grocery_financial_rf <- randomForest(ABC_Class ~ . - ABC_Class,
                       data = train_financial_quality, mtry = 2,
                       importance = TRUE, ntree = 1000, maxnodes = 4)
rf_predict_financial <- predict(grocery_financial_rf, test_financial_quality)
confusionMatrix(rf_predict_financial, as.factor(test_financial_quality$ABC_Class))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   A   B   C
##          A   3   0   0
##          B  31  50   0
##          C   5   9 102
##
## Overall Statistics
##
##                Accuracy : 0.775
##                  95% CI : (0.7108, 0.8309)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : 1.057e-14
##
##                   Kappa : 0.6133
##
##  Mcnemar's Test P-Value : 9.253e-10
##
## Statistics by Class:
##
##                      Class: A Class: B Class: C
## Sensitivity           0.07692   0.8475   1.0000
## Specificity           1.00000   0.7801   0.8571
## Pos Pred Value        1.00000   0.6173   0.8793
## Neg Pred Value        0.81726   0.9244   1.0000
## Prevalence            0.19500   0.2950   0.5100
## Detection Rate        0.01500   0.2500   0.5100
## Detection Prevalence  0.01500   0.4050   0.5800
## Balanced Accuracy     0.53846   0.8138   0.9286
```

```r
barplot(grocery_financial_rf$importance[,3])
```

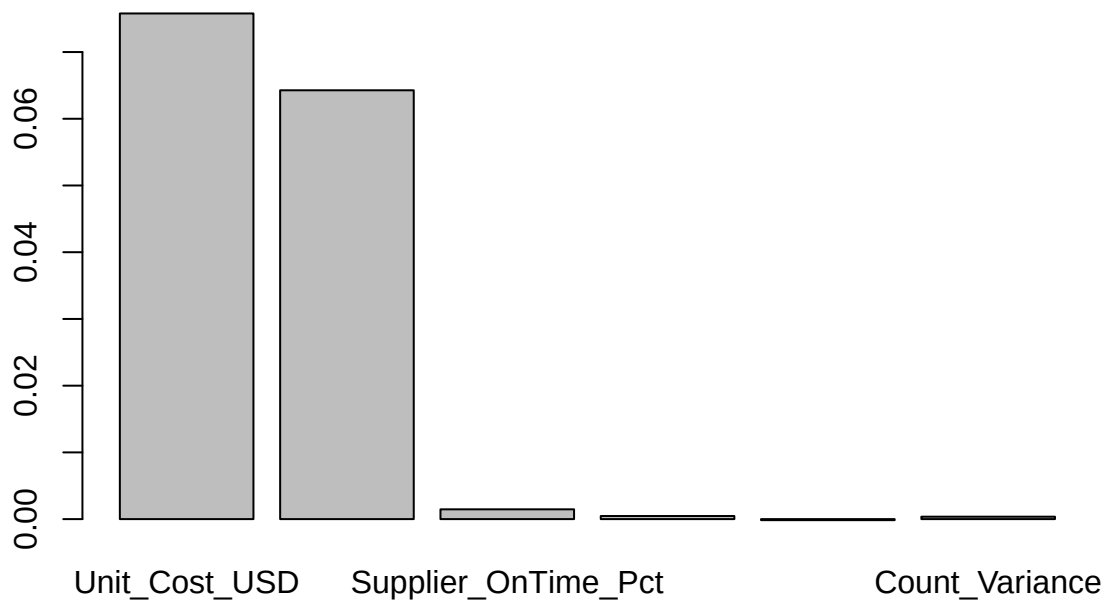



```
grocery_financial_rf <- randomForest(ABC_Class ~ . - ABC_Class - Total_Inventory_Value_USD,
                      data = train_financial_quality, mtry = 2,
                      importance = TRUE, ntree = 1000, maxnodes = 4)
rf_predict_financial <- predict(grocery_financial_rf, test_financial_quality)
confusionMatrix(rf_predict_financial, as.factor(test_financial_quality$ABC_Class))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  A  B  C
##          A  1  1  1
##          B 32 37 36
##          C  6 21 65
##
## Overall Statistics
##
##                Accuracy : 0.515
##                  95% CI : (0.4435, 0.5861)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : 0.472
##
##                   Kappa : 0.2018
##
##  Mcnemar's Test P-Value : 5.483e-08
##
```

```
## Statistics by Class:
##
##                      Class: A Class: B Class: C
## Sensitivity           0.02564   0.6271   0.6373
## Specificity           0.98758   0.5177   0.7245
## Pos Pred Value        0.33333   0.3524   0.7065
## Neg Pred Value        0.80711   0.7684   0.6574
## Prevalence            0.19500   0.2950   0.5100
## Detection Rate        0.00500   0.1850   0.3250
## Detection Prevalence  0.01500   0.5250   0.4600
## Balanced Accuracy     0.50661   0.5724   0.6809
```

```r
barplot(grocery_financial_rf$importance[,3])
```




```r
grocery_financial_rf <- randomForest(ABC_Class ~ Total_Inventory_Value_USD,
                      data = train_financial_quality, mtry = 2,
                      importance = TRUE, ntree = 1000, maxnodes = 4)
```

```
## Warning in randomForest.default(m, y, ...): invalid mtry: reset to within valid
## range
```

```r
rf_predict_financial <- predict(grocery_financial_rf, test_financial_quality)
confusionMatrix(rf_predict_financial, as.factor(test_financial_quality$ABC_Class))
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   A   B   C
##          A  39   0   0
##          B   0  59   0
##          C   0   0 102
##
## Overall Statistics
##
##                Accuracy : 1
##                  95% CI : (0.9817, 1)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: A Class: B Class: C
## Sensitivity             1.000    1.000     1.00
## Specificity             1.000    1.000     1.00
## Pos Pred Value          1.000    1.000     1.00
## Neg Pred Value          1.000    1.000     1.00
## Prevalence              0.195    0.295     0.51
## Detection Rate          0.195    0.295     0.51
## Detection Prevalence    0.195    0.295     0.51
## Balanced Accuracy       1.000    1.000     1.00
```

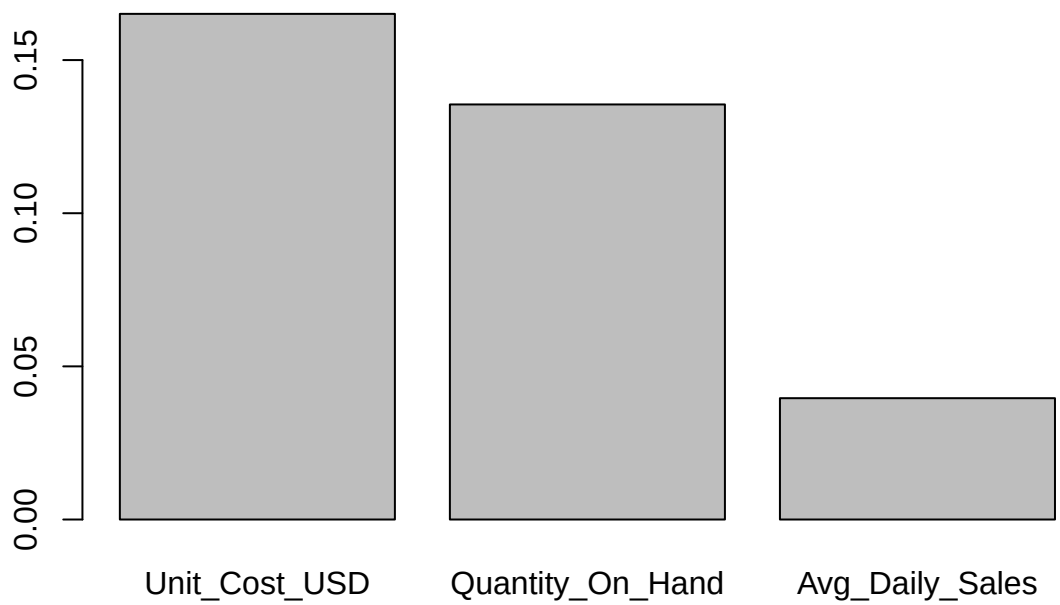
```r
#barplot(grocery_financial_rf$importance[,3])
```

```r
grocery_tester_rf <- randomForest(ABC_Class ~ Unit_Cost_USD + Quantity_On_Hand + Avg_Daily_Sales,
                      data = train, mtry = 2,
                      importance = TRUE, ntree = 1000, maxnodes = 4)
rf_predict_tester <- predict(grocery_tester_rf, test)
confusionMatrix(rf_predict_tester, as.factor(test$ABC_Class))
```

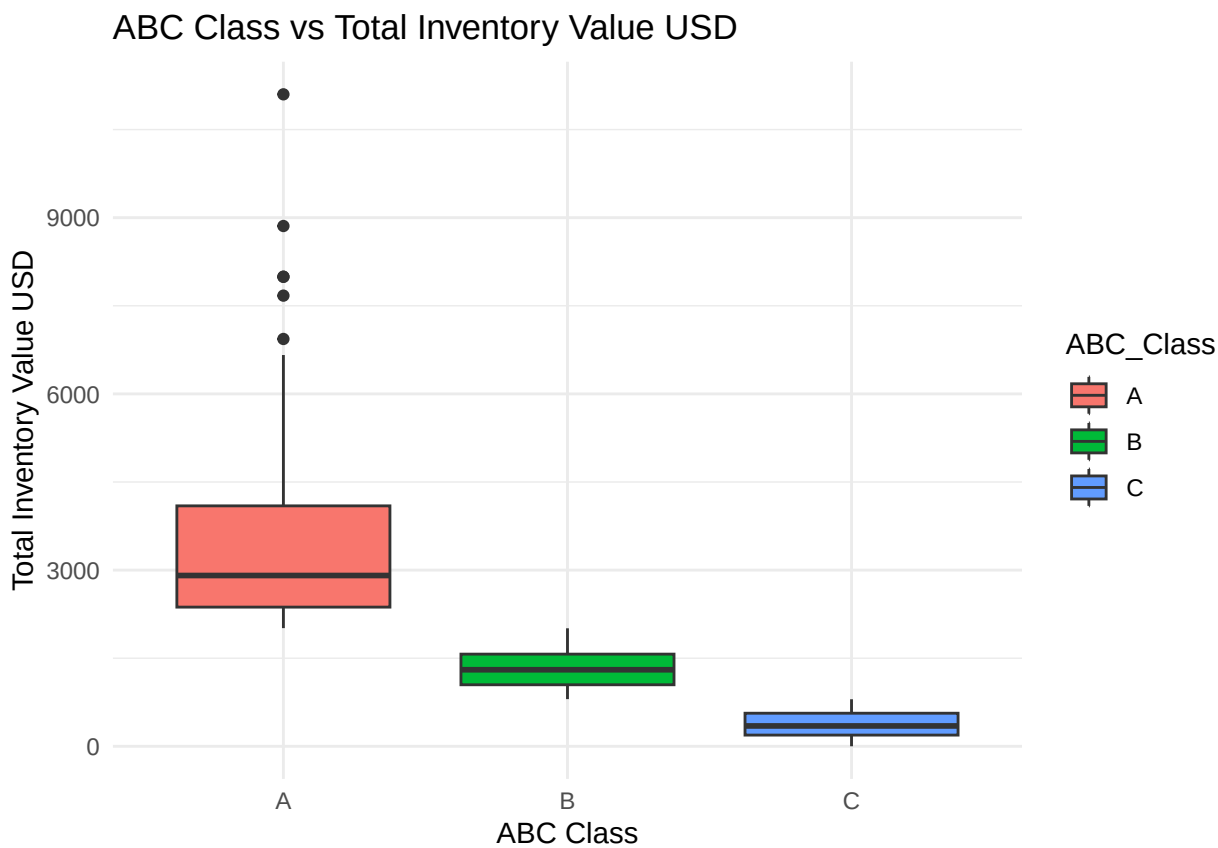
```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  A  B  C
##          A 31 15  3
##          B  6 15  2
##          C  2 29 97
##
## Overall Statistics
##
##                Accuracy : 0.715
##                  95% CI : (0.6471, 0.7764)
##     No Information Rate : 0.51
##     P-Value [Acc > NIR] : 2.836e-09
```

```
##
##                   Kappa : 0.5185
##
##  Mcnemar's Test P-Value : 4.464e-06
##
## Statistics by Class:
##
##                      Class: A Class: B Class: C
## Sensitivity            0.7949   0.2542   0.9510
## Specificity            0.8882   0.9433   0.6837
## Pos Pred Value         0.6327   0.6522   0.7578
## Neg Pred Value         0.9470   0.7514   0.9306
## Prevalence             0.1950   0.2950   0.5100
## Detection Rate         0.1550   0.0750   0.4850
## Detection Prevalence   0.2450   0.1150   0.6400
## Balanced Accuracy      0.8415   0.5987   0.8173
```

```r
barplot(grocery_tester_rf$importance[,3])
```




```r
ggplot(grocery, aes(x = ABC_Class, y = Total_Inventory_Value_USD, fill = ABC_Class)) +
  geom_boxplot() +
  labs(title = "ABC Class vs Total Inventory Value USD",
       x = "ABC Class",
       y = "Total Inventory Value USD") +
  theme_minimal()
```



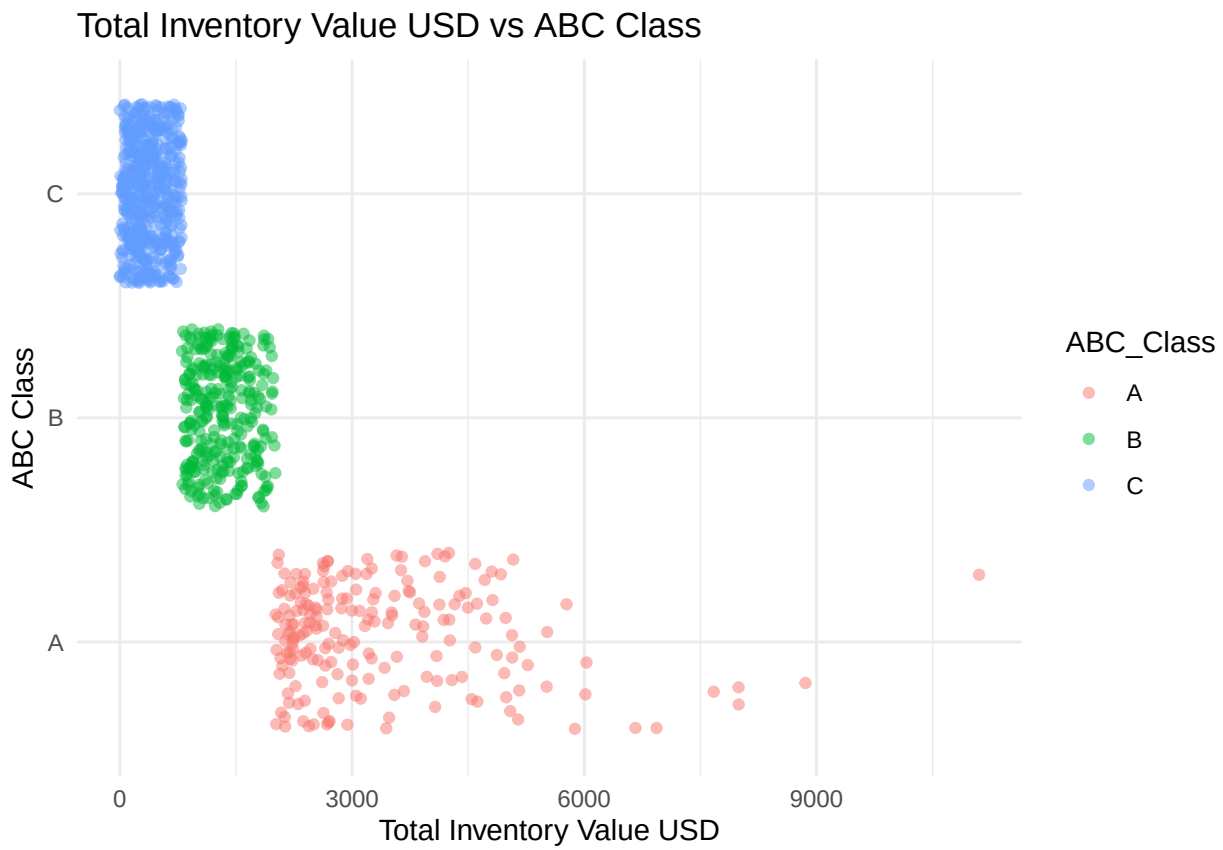

```r
model_logit <- multinom(
  ABC_Class ~ Total_Inventory_Value_USD,
  data = train)
```

```
## # weights:  9 (4 variable)
## initial  value 878.889831
## iter  10 value 101.532338
## iter  20 value 14.877463
## iter  30 value 10.085841
## iter  40 value 8.684546
## iter  50 value 6.630872
## iter  60 value 6.394047
## iter  70 value 5.984196
## iter  80 value 5.837901
## iter  90 value 5.535443
## iter 100 value 5.348749
## final  value 5.348749
## stopped after 100 iterations
```

```r
pred_logit <- predict(model_logit, test)
mean(pred_logit == test$ABC_Class)
```

```
## [1] 1
```

```r
ggplot(grocery, aes(x = Total_Inventory_Value_USD, y = ABC_Class, color = ABC_Class)) +
  geom_jitter(alpha = 0.5) +
    labs(title = "Total Inventory Value USD vs ABC Class",
       y = "ABC Class",
       x = "Total Inventory Value USD") +
  theme_minimal()
```




```r
grocery %>%
  group_by(ABC_Class) %>%
  summarise(
    'Lowest Value' = min(Total_Inventory_Value_USD, na.rm = TRUE),
    'Highest Value' = max(Total_Inventory_Value_USD, na.rm = TRUE))
```

```
## # A tibble: 3 x 3
##   ABC_Class `Lowest Value` `Highest Value`
##   <fct>              <dbl>           <dbl>
## 1 A                  2012.          11100
## 2 B                   803.           2009.
## 3 C                     0             801.
```

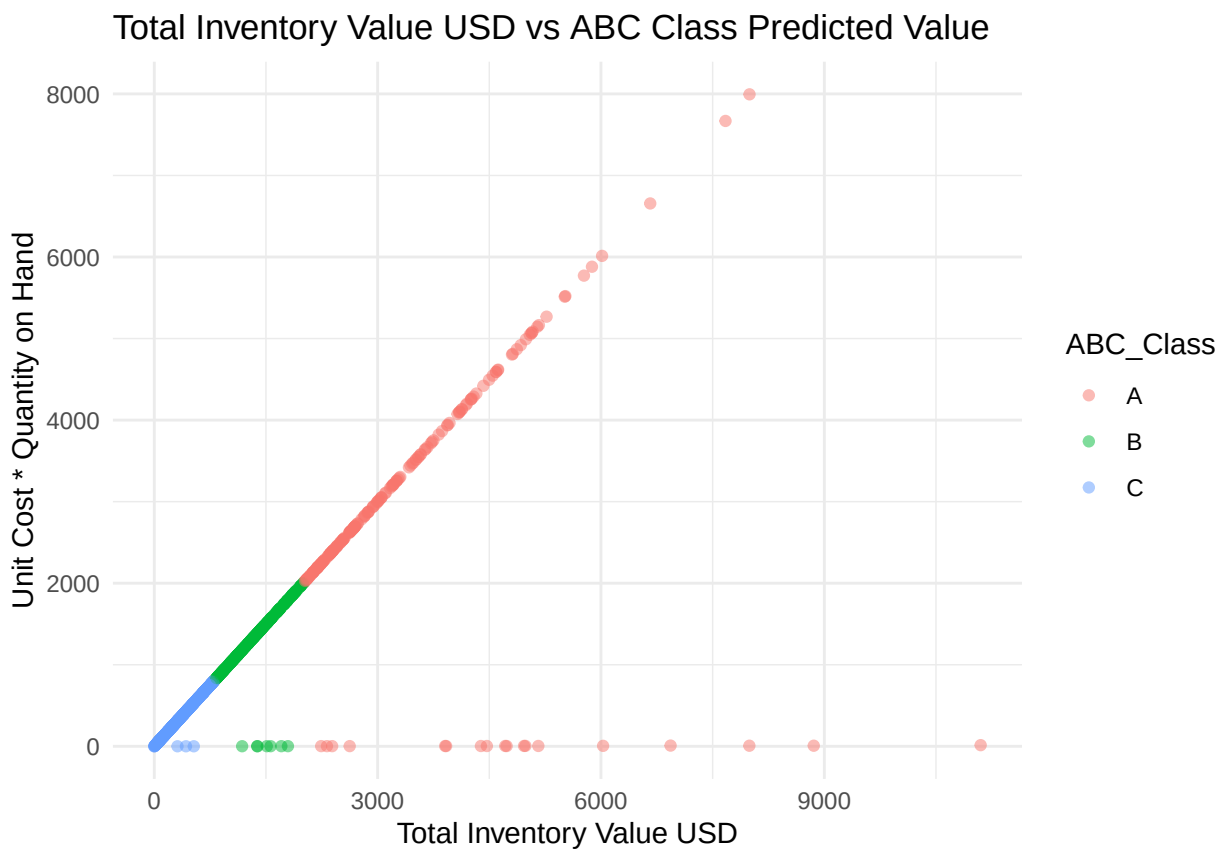
```r
abc_class_solver <- grocery[c('Quantity_On_Hand', 'Quantity_Reserved', 'Quantity_Committed', 'Unit_Cost_
abc_class_solver['verification1'] <- abc_class_solver['Quantity_On_Hand'] * abc_class_solver['Unit_Cost_
abc_class_solver['verification2'] <- abc_class_solver['Quantity_On_Hand'] * abc_class_solver['Last_Purc
abc_class_solver['verification3'] <- abc_class_solver['Avg_Daily_Sales'] * abc_class_solver['Unit_Cost_
head(abc_class_solver)
```

```
##   Quantity_On_Hand Quantity_Reserved Quantity_Committed Unit_Cost_USD
## 1              359               100                 36          5.81
## 2              314                64                  0          1.45
## 3              485                28                 62          1.86
## 4              281                92                 49         14.50
## 5              235                63                 33          9.46
## 6              446                66                 31          5.16
##   Last_Purchase_Price_USD Avg_Daily_Sales ABC_Class Total_Inventory_Value_USD
## 1                    5.71           28.57         A                   2084.25
## 2                    1.33           34.99         C                    456.71
## 3                    1.69           36.55         B                    902.63
## 4                   14.12           25.49         A                   4073.15
## 5                   10.31           17.05         A                   2222.91
## 6                    5.86           32.81         A                   2303.14
##   verification1 verification2 verification3
## 1       2085.79       2049.89      165.9917
## 2        455.30        417.62       50.7355
## 3        902.10        819.65       67.9830
## 4       4074.50       3967.72      369.6050
## 5       2223.10       2422.85      161.2930
## 6       2301.36       2613.56      169.2996
```

```
abc_class_solver$percent_error <- round(abs(abc_class_solver$verification1 - abc_class_solver$Total_Inve
head(abc_class_solver)
```

```
##   Quantity_On_Hand Quantity_Reserved Quantity_Committed Unit_Cost_USD
## 1              359               100                 36          5.81
## 2              314                64                  0          1.45
## 3              485                28                 62          1.86
## 4              281                92                 49         14.50
## 5              235                63                 33          9.46
## 6              446                66                 31          5.16
##   Last_Purchase_Price_USD Avg_Daily_Sales ABC_Class Total_Inventory_Value_USD
## 1                    5.71           28.57         A                   2084.25
## 2                    1.33           34.99         C                    456.71
## 3                    1.69           36.55         B                    902.63
## 4                   14.12           25.49         A                   4073.15
## 5                   10.31           17.05         A                   2222.91
## 6                    5.86           32.81         A                   2303.14
##   verification1 verification2 verification3 percent_error
## 1       2085.79       2049.89      165.9917       0.07389
## 2        455.30        417.62       50.7355       0.30873
## 3        902.10        819.65       67.9830       0.05872
## 4       4074.50       3967.72      369.6050       0.03314
## 5       2223.10       2422.85      161.2930       0.00855
## 6       2301.36       2613.56      169.2996       0.07729
```

```
ggplot(abc_class_solver, aes(x = Total_Inventory_Value_USD, y = verification1, color = ABC_Class)) +
  geom_jitter(alpha = 0.5) +
    labs(title = "Total Inventory Value USD vs ABC Class Predicted Value",
       y = "Unit Cost * Quantity on Hand",
       x = "Total Inventory Value USD") +
  theme_minimal()
```


Total Inventory Value USD vs ABC Class Predicted Value
Unit Cost * Quantity on Hand
8000
6000
4000
2000
0
0
3000
6000
9000
Total Inventory Value USD
ABC_Class
A
B
C